Quando a orquestração de agentes de IA compensa?

Um estudo empírico com o NarraTer

Andamento do Trabalho de Conclusão de Curso

Gabriel Cortez

12 de novembro de 2026

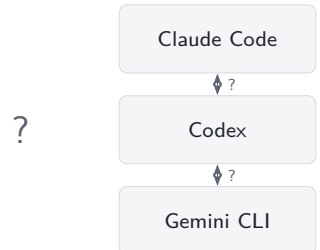
Roteiro

1. De onde o trabalho partiu, e como a pergunta mudou
2. O que a literatura diz sobre orquestrar agentes
3. O instrumento: o NarraTer e como ele produz evidência
4. Fase exploratória: tornar a orquestração confiável e medível
5. O experimento central: equipe contra agente sozinho, em duas tarefas
6. Da evidência à ferramenta: o portão, o gerador de equipes e o aceite
7. Limitações, próximos passos e perguntas ao orientador

De onde partimos

Agentes de código vivem em terminais isolados

- Os agentes de programação mais usados hoje são **programas de terminal**: Claude Code, Codex, Gemini CLI, OpenCode.
- Cada um é de um fornecedor, com sua interface e seus formatos internos.
- Eles não conversam entre si. Quem passa contexto de um para outro é a pessoa, copiando e colando.
- Quando um fornecedor oferece times de agentes, só funciona **dentro do próprio produto**.



Como a pergunta mudou

julho — outubro — 15/10 — 16/10

Uma ferramenta

Fazer agentes de fornecedores diferentes (Claude, Gemini, Codex) conversarem num canvas.

Modelo cooperativo: só funcionou com o Claude.

Um protocolo

O terminal como API universal (D01). Para funcionar, foi preciso **medir**: quando o agente terminou? A resposta chega inteira?

Um benchmark

Sete versões, 24 rodadas. Todas tiraram nota máxima; o que mudava era o **custo de coordenar**.

Uma pergunta

Se a nota não muda e só o custo muda: **um agente sozinho não faria igual?** Quando a orquestração compensa?

Construir a ferramenta antes de fechar a teoria foi uma **fase exploratória**: foi ela que revelou a pergunta certa e produziu o instrumento para respondê-la.

Pergunta de pesquisa

Quando a orquestração de vários agentes de IA compensa, em relação a **um agente sozinho**, em tarefas de programação? E como uma ferramenta pode **decidir** isso?

Q1 Em que tipo de tarefa uma equipe ganha de um agente sozinho, em **nota, tempo e custo**?

Q2 **De onde vem** o custo de coordenar: do protocolo, da forma da equipe ou do tamanho das partes?

Q3 Que **regra** uma ferramenta deve seguir para montar, ou não montar, uma equipe?

Pré-requisito, já resolvido na fase exploratória: uma orquestração entre agentes de qualquer fornecedor que saiba quando o agente terminou e receba a resposta inteira.

O que a literatura diz

Fonte	Achado
MAST (Cemri et al., 2025)	14 modos de falha em sistemas multiagente: projeto do sistema ∼44%, desalinhamento entre agentes ∼32%, **verificação** ∼23,5%.
Kim et al. (Google/MIT, 2025)	180 configurações: coordenar ganha 80,8% em tarefas **paralelizáveis** e perde 39–70% nas **sequenciais**; acima de ∼45% de acerto do agente sozinho, coordenar não compensa.
Vals.ai (2026)	Equipes construindo apps: custo de 1,8× a 5,1×; ganho significativo em 1 de 4 configurações.
Tran e Kiela (2026)	Com o mesmo orçamento de tokens, o agente sozinho empata ou ganha.
Anthropic (2025)	Multiagente vale em pesquisa ampla e paralela, a ∼15× os tokens; código é um encaixe ruim.
Cognition (2025–26)	"Não construa multiagentes": a escrita deve ficar numa linha só. O que funciona é um **revisor com contexto limpo**.

[citado] Números principais conferidos na fonte (`docs/pesquisa/orquestracao.md`).

[inferido] Hipótese: em tarefas que um agente forte resolve sozinho, a equipe só acrescenta custo.

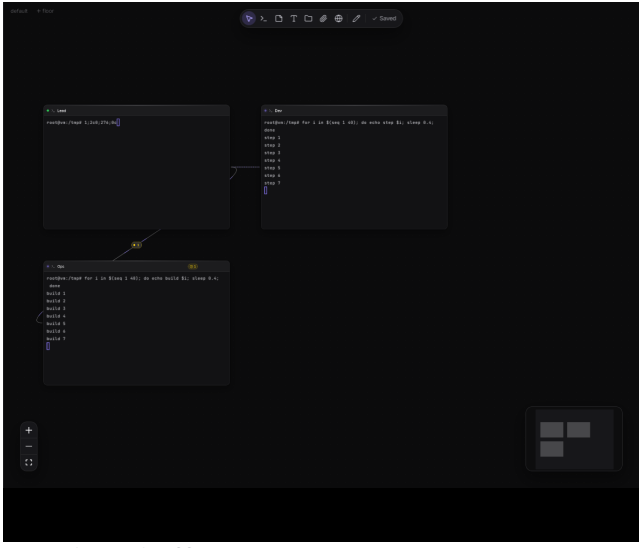
Ferramentas relacionadas

Projeto	Como sabe que o agente terminou	Como obtém a resposta	Exige cooperação?
Maestri (macOS)	tela	tela	não
Orca	*hooks* em ∼25 CLIs + regras de tela em JSON + título da janela	ferramenta `worker-read`, blocos de 1.200 caracteres	sim
CLI Agent Orchestrator (AWS)	expressões regulares por CLI sobre a tela	tela	não
Agent Teams (Claude Code)	evento interno do programa	mensagem empurrada	só Claude
NarraTer	tela + título + *transcript* quando existe	*transcript*, ou tela	**não**

Estudos em `docs/pesquisa/` (código-fonte lido em versões e *commits* registrados). A literatura acadêmica está no slide anterior.

O instrumento: NarraTer

O NarraTer: um canvas de terminais



- Aplicativo para Linux (Tauri 2: Rust + React).
- Cada **nó** é um terminal real (um PTY).
- Cada **aresta** é uma permissão: quem pode falar com quem.
- Os agentes conversam por uma CLI, `narrater`, que o app instala.
- O estado de cada terminal (trabalhando, pronto, precisa de você) vem da tela.

Exemplo: um líder delega a um desenvolvedor

O Narrador (líder) roda no seu terminal:

```
$ narrater ask DevA "Implemente a
    conversão de moedas e rode os testes.
    Responda com o resultado."
```

E recebe de volta a resposta final do DevA:

```
Implementei a funcionalidade multimoeda
e os testes passam (43 testes, OK). As
alterações estão em pedidos/cambio.py,
modelo.py ...
```

O DevA não instalou nada nem conhece protocolo: o pedido foi digitado no terminal dele.



A tese central: o terminal é a API universal (decisão D01)

Julho: modelo cooperativo

- Servidor MCP, *prompt* de sistema extra, *hook* de parada.
- O alvo responde por ID com uma ferramenta própria.
- Só funcionava com o Claude Code. Um alvo que não conhece o protocolo trava o pedido até o *timeout*.

Outubro: modelo agnóstico

- O estado do turno é lido da **tela renderizada** (emulador vt100 sem interface).
- O pedido é digitado quando o alvo está ocioso.
- Instruções chegam por arquivo (`CLAUDE.md`, `AGENTS.md`), que toda CLI lê.
- Qualquer programa que roda num terminal participa.

Regra derivada (D05/D06)

Cooperação do agente só entra como **opcional que melhora o resultado**. Se o agente não cooperar, tudo continua funcionando pela tela.

Registro de decisões com evidência classificada (D09)

Cada decisão com alternativa real vira uma entrada em `docs/decisoes.md` (24 até agora), com seis campos:

- contexto, decisão;
- alternativas recusadas e por quê;
- evidência **rotulada**: [medido] [observado] [citado] [inferido]
- consequência;
- **condição que derrubaria a decisão**.

O arquivo só cresce: uma decisão revista ganha entrada nova que cita a antiga.

```
## D20 · A resposta do ask e o fim do turno vêm
##       do transcript do agente quando ele existe

Alternativas consideradas.
- Scrollback do emulador: o Claude Code roda em
  tela alternativa, o histórico é vazio [medido]
- Usar só o end_turn: o bloco de pensamento traz
  end_turn 15 s antes do texto [observado]

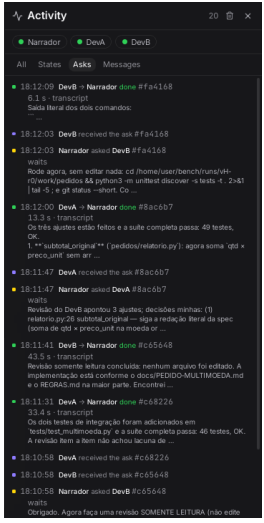
Evidência.
- [medido] acha a resposta certa em 126 de 126
  pedidos das rodadas C a G
- [medido] asks prontos antes da hora:
  D 4 de 27, E 12 de 31, F e G 0 de 40

Revisar se: o Claude Code mudar o formato do
transcript; o benchmark voltar a mostrar asks
antes da hora ou respostas trocadas.
```

Regra de método: medir contra o programa real (D12)

- Toda regra que lê a tela de um agente é testada contra o **programa real**, nunca contra uma tela lembrada.
- Cada execução medida é gravada (asciicast) e vira **teste de regressão**: `tests/transcripts/` tem gravações reais do Claude Code (resposta curta, longa, ferramenta silenciosa, diálogo de permissão...).
- A verdade vem de um **oráculo independente da tela**, ligado só no avaliador:
  - shell: o próprio comando grava um *timestamp* ao terminar;
  - Claude Code: *hooks* `Stop` e `PermissionRequest`, configurados só na medição.
- O app continua sem *hooks*: o oráculo mede, não participa (D01 preservada).

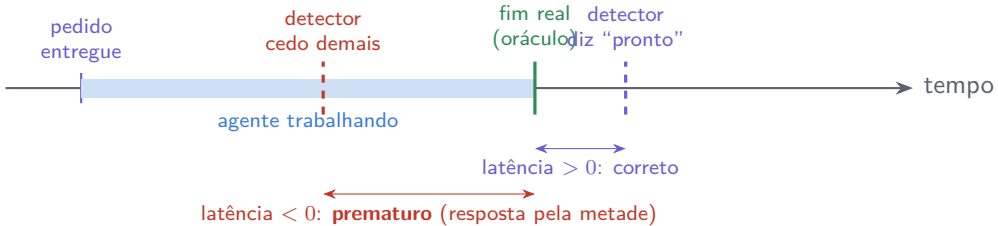
Verificabilidade: o painel Activity (D21)



- Cada estado, pedido, entrega e resposta é gravado **pelo backend**, no momento em que acontece (`activity.jsonl`).
- Nenhum agente escreve nesse registro: é o fato, não a palavra da IA.
- Cada resposta mostra a duração e a **origem** (transcript ou tela).
- Serve para auditar uma orquestração depois, e para explicar falhas como o ask antes da hora.

Fase exploratória: tornar a orquestração confiável e medível

Método: latência contra o oráculo



- Detector = o próprio código do app (`screen.rs`), no mesmo ciclo de 200 ms.
- Cenários: shell silencioso, com saída contínua, rajada + silêncio (direto e dentro do tmux); Claude Code com resposta curta, ferramenta silenciosa, resposta longa e pedido de permissão.
- Ferramenta: `cargo run -release -example turn_eval - -reps N [-claude]`

Resultados: antes e depois das correções (D10, D11)

	Antes		Depois	
Cenário	corretos/prem. (n)	latência	corretos/prem. (n)	latência
shell direto (3 cenários)	9 / 0 (9)	0,6–0,8 s	30 / 0 (30)	0,6–0,8 s
tmux, saída contínua	3 / 0 (3)	0,76 s	10 / 0 (10)	0,75–0,78 s
tmux, comando silencioso	0 / 3 (3)	−3,2 s	10 / 0 (10)	0,6–0,8 s
tmux, rajada + silêncio	0 / 3 (3)	−2,2 s	10 / 0 (10)	0,8 s
Claude, turnos normais	9 / 0 (9)	mediana 1,36 s	15 / 0 (15)	mediana 0,55 s
Claude, permissão	nunca detectado	—	3 / 0 (3)	0,1–0,2 s

[medido] Claude, turnos normais: antes n=9, depois n=15, faixas sem sobreposição (Mann–Whitney exato, $p \approx 1{,}5 \times 10^{-6}$). tmux silencioso: 6/6 prematuros antes, 0/20 depois (Fisher exato, $p \approx 4{,}3 \times 10^{-6}$).

Ressalvas: uma máquina, uma versão do Claude Code (2.1.299, modelo Haiku); Codex e Gemini não medidos.

Desenho experimental (D19)

Tarefa: um repositório Python inventado (`pedidos`, ∼300 linhas) com regras de negócio próprias.

- implementar pedidos em várias moedas (atravessa 3 módulos);
- corrigir 2 defeitos descritos só como reclamação de cliente;
- manter e escrever testes.

Julgamento: 31 **testes escondidos** que os agentes nunca veem, em 4 níveis. Solução de referência: 31/31. Repositório inicial: 5/31.

Uma calculadora foi recusada: o modelo sabe o exercício de cor.

Equipe fixa:

- Narrador (líder, não edita): Claude Opus 5.5
- DevA: Claude Sonnet 5.5
- DevB: Claude Haiku 5.5

Controles:

- toda rodada começa **zerada** (HOME, sessões, tmux, pasta);
- mesma versão do Claude Code em todas;
- **ordem contrabalançada** (o *cache* do servidor dura 1 h);
- 3 rodadas por versão;
- métricas tiradas das transcrições, não do que o agente diz.

As versões comparadas: cada uma é um *commit* real

	Commit	O que muda
A	`e708d7b`	Cooperativo (julho): MCP, *prompt* de sistema, *hook*. Só Claude. O dev envia o relatório inteiro.
B	`16cee0e`	Agnóstico (D01): CLI + *skill*; o `ask` devolve a tela do alvo.
C	`95fb11a`	`ask` assíncrono e durável.
D	`db3d535`	Leitor de estado novo (Estudo 1) e *statusline*.
E	`8e2e5fa`	A resposta vem do *transcript* do agente, não da tela. **E-sl0**: E sem *statusline* (ablação).
F	`26e3c06`	O *transcript* também decide se o turno acabou.
G	`c6ba63e`	Cada `ask` acha o próprio turno (corrige pedidos paralelos).

A–D mediram a história do projeto. E–G foram criadas **a partir do diagnóstico** de A–D, cada uma corrigindo uma falha vista nas rodadas da anterior.

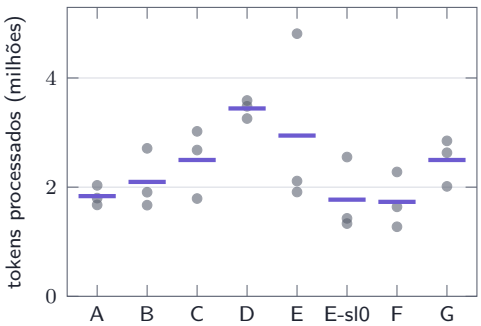
O resultado que mudou a pergunta: a qualidade bateu no teto

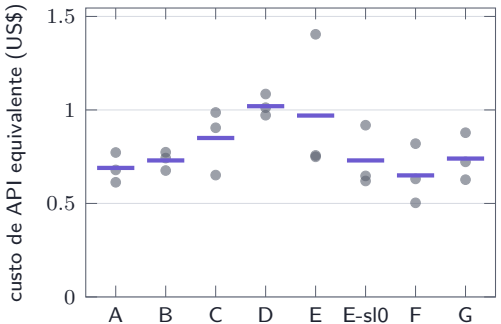
24/24

rodadas de equipe passaram em **31 de 31** testes escondidos

- [medido] Todas as versões, todas as rodadas, nota máxima. O líder nunca editou um arquivo.
- O que separava as versões era só o **custo**: tokens, tempo, quantas vezes o líder voltava aos devs.
- A comparação era sempre **equipe contra equipe**. Faltava o grupo de controle: **um agente sozinho** na mesma tarefa.

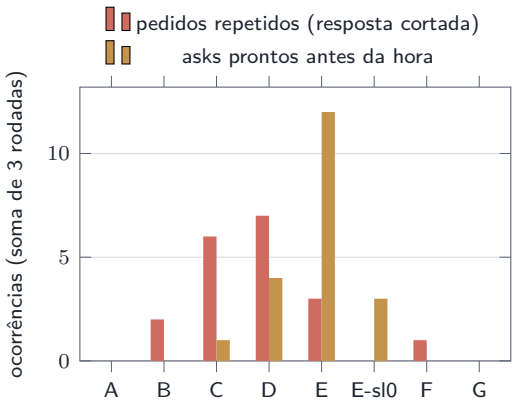
O custo de coordenar mudou entre versões





Cada ponto é uma rodada; o traço é a média. [medido] De A até D o custo **subiu** (1,8 → 3,4 milhões de tokens; US$ 0,69 → 1,02). Com F e G ele voltou ao nível da A (US$ 0,65 e 0,74), **sem** prender o NarraTer a um fornecedor.

Por que D custou mais? O mecanismo encontrado



"Antes da hora" só é medível a partir da C.

[medido] A latência do protocolo é de segundos (entrega 0,2–0,5 s; detecção 1,6–3,1 s). A duração cresce em **minutos**: o que cresce é o número de pedidos do líder (3–5 na A, 7–10 na D).

Duas falhas fazem o líder pedir de novo:

1. **resposta cortada**: a tela mostra ∼13 linhas, e o relatório do dev é maior;
2. **ask antes da hora**: o leitor dava o turno por encerrado 4–5 s após a entrega, antes de o agente começar.

[medido] Na rodada E2, 8 de 16 asks voltaram cedo; o líder deixou de confiar no `ask`: 14 pedidos e 10 minutos.

Exemplo real: o `end_turn` que mente

Trecho do *transcript* da rodada E2 (hoje é um teste em `tests/transcripts/`), resumido:

```
15:40:32.7  user       "[narrater from Narrador]: Projeto: .../pedidos. NÃO altere nada..."
15:40:35.3  assistant  tool_use  Bash
15:40:37.3  user       tool_result
15:40:47.7  assistant  thinking            stop_reason = end_turn     <- parece o fim
15:41:03.1  assistant  text "Não alterei nada. Li REGRAS.md, ..."  stop_reason = end_turn
15:41:03.2  system     turn_duration                                    <- fim de verdade
```

- [observado] O Claude Code grava cada bloco da mensagem como um registro, e todos levam o `stop_reason`. O pensamento já diz `end_turn` **15 s antes** de a resposta existir.
- O fim confiável é o registro `system/turn_duration`. A versão F passou a esperar por ele.
- É formato interno do fornecedor e pode mudar: por isso a tela continua como reserva e o benchmark é o alarme.

As falhas do protocolo foram a zero

	A (cooperativo)	D	E	F	G
Asks prontos antes da hora	—	4/27	12/31	**0/16**	**0/24**
Pedidos repetidos (por rodada)	0, 0, 0	1, 5, 1	0, 2, 1	0, 1, 0	**0, 0, 0**
Pedidos do líder aos devs	3–5	7–10	5–14	3–7	3–6
Tokens processados (milhões)	1,84	3,44	2,94	1,73	2,50
Custo médio (US$)	0,69	1,02	0,97	0,65	0,74
Prende a um fornecedor?	sim	não	não	não	não

- [medido] D contra G: sem sobreposição em tokens, custo e pedidos (Mann–Whitney exato, U = 9, $p = 0{,}10$, o menor p possível com 3 contra 3).
- [medido] O leitor de *transcript* acha a resposta certa em 126 de 126 pedidos reais das rodadas C–G.
- [inferido] O que ainda separa G de A é o ritmo dos agentes, não a coordenação: na G3, 243 s foram o DevB (Haiku) trabalhando sem parar.

O experimento central: equipe contra agente sozinho

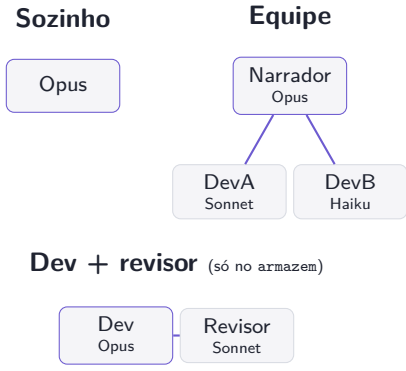
Desenho: o braço “sozinho”

Mesmo benchmark, mesma tarefa, mesmos testes escondidos, mesmas condições (rodada zerada, 3 por montagem). Muda só **quem trabalha**:

- **Sozinho**: um terminal, Claude Opus 5.5, o mesmo pedido.
- **Equipe**: Narrador (Opus) + DevA (Sonnet) + DevB (Haiku), versões F e G do protocolo, já sem as falhas da fase exploratória.

Duas tarefas:

- `pedidos`: funcionalidade que atravessa 3 módulos + 2 defeitos (31 testes);
- `armazem`: **4 módulos independentes**, feita para a equipe ganhar (46 testes, 12 de regras sutis).



Tarefa `pedidos`: o agente sozinho venceu

	Sozinho	Equipe G	Equipe F
Testes escondidos	31/31	31/31	31/31
Duração (min)	**1,6**	3,9	2,8
Tokens (milhões)	**0,36**	2,50	1,73
Custo (US$)	**0,55**	0,74	0,65
Chamadas à API	**8**	49	38

Médias de 3 rodadas por montagem.

- [medido] Contra a G, o sozinho vence em **tempo, tokens, chamadas e custo**, sem sobreposição (Mann–Whitney, $p = 0{,}10$, o mínimo com 3 contra 3).
- [medido] Só o Narrador da G, **apenas coordenando**, custou US$ 0,32–0,41: de 60% a 75% do Opus sozinho fazendo tudo.
- [inferido] Nesta tarefa, a equipe só acrescenta custo. Faltava uma tarefa em que ela *deveria* ganhar.

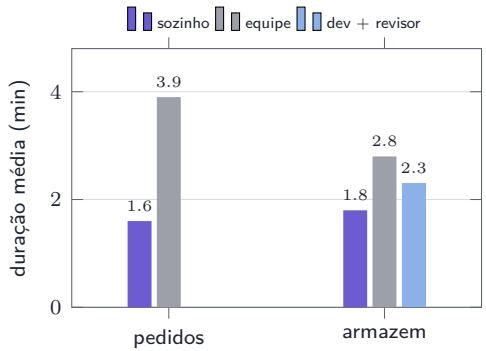
Tarefa `armazem`: previsões registradas *antes* de rodar

Escritas a partir da literatura e gravadas num *commit* (52688a3) antes da primeira rodada válida:

	Previsão (fonte)	O que aconteceu	Veredito
P1	A equipe paralela termina antes (Kim et al.: +80,8% em tarefas paralelas)	2,8 contra 1,8 min, sem sobreposição	**Refutada**
P2	A equipe gasta mais (Vals.ai: 1,8–5,1×)	4,2× os tokens; custo igual (modelos baratos)	Em parte
P3	O revisor melhora a nota nas regras sutis (Cognition)	Nota igual, 36% mais caro, aprovou tudo	Não testável
P4	No teto, ninguém melhora a nota (Kim et al.)	46/46 nas 9 rodadas	**Confirmada**
P5	Os erros da equipe virão do Haiku (MAST)	A equipe não errou; o Haiku foi o gargalo de *tempo*	Não se aplica

Registrar a previsão antes protege contra explicar o resultado depois de vê-lo.

Por que a equipe perdeu numa tarefa paralelizável



No `pedidos`, a equipe é a G; não houve braço dev + revisor.

[medido] O protocolo **não falhou**: 0 de 15 *asks* prematuros, 0 respostas cortadas. O tempo se perdeu na **forma da equipe**:

- o Haiku levou 109–127 s nos seus módulos, **o dobro** do Sonnet;
- os dois devs trabalharam juntos só 39–59 s;
- o Narrador ficou sozinho 39–49 s, o **custo fixo de coordenar**.

[inferido] Ser paralelizável não basta: cada parte precisa valer bem mais que $\sim$1 min de coordenação, e os membros precisam ter a mesma velocidade.

Da evidência à ferramenta

O padrão passa a ser um agente só (D22)

Quando alguém decide como atacar uma tarefa (a pessoa, o líder ou o gerador), a resposta padrão é **um agente só**. Montar equipe exige pelo menos **um motivo explícito**:

1. duas ou mais subtarefas independentes, com arquivos separados e grandes o bastante;
2. um **verificador com contexto limpo**, de preferência de outro fornecedor;
3. um contexto que não cabe num agente.

O motivo fica registrado numa nota no canvas.

Por que não "nunca orquestrar"? A literatura mostra ganho real em leitura paralela (Kim et al.) e com revisor de contexto limpo (Cognition). É para esses casos que a ferramenta existe.

Revisar se: um formato medido no benchmark ganhar do agente sozinho. Ele vira o padrão sugerido.

Este é o principal resultado prático até aqui: uma regra de decisão com evidência medida e citada, e com a condição que a derrubaria.

O gerador de equipes (D23) e o aceite verificado (D24)

Gerador: escolhe entre **formatos conhecidos**, não inventa equipes.

```
narrater team create --template dev-reviewer \
  --task "Implementar a exportação em CSV" \
  --reason "revisor com contexto limpo" \
  --check "python3 -m unittest -q"
```

- formatos: `solo`, `dev-reviewer`, `dev-qa`, `readers`, `parallel-devs`, cada um com "quando compensa" e o custo medido;
- **o portão**: sem `-reason`, é recusado; `solo` não cria terminal;
- cria terminais com modelo e papel, arestas, uma nota com a tarefa e o critério de aceite;
- **Automatic teams**: uma chave do usuário, desligada por padrão.

A literatura que funciona (AFlow, MaAS, MasRouter) escolhe entre peças conhecidas.

Aceite: "pronto" é verificado pelo ambiente, não pela palavra do agente.

```
narrater ask "Dev" "..." --check "make test"
```

- quando o turno termina, o **backend** roda o comando;
- saída 0: "Acceptance check passed"; senão, o pedido termina como `failed`, com a saída;
- [citado] verificação é $\sim$23,5% das falhas (MAST), e juízes LLM não detectam o "pronto" falso (AUROC $\leq$ 0,65);
- não exige cooperação do agente (D01 preservada).

[observado] Ambos verificados de ponta a ponta no app real.

Limitações e próximos passos

Ameaças à validade

- **Amostra pequena**: 3 rodadas por montagem; o menor p possível é 0,10. Nada é significativo a 5%.
- **Teto**: em 2 tarefas e 18 rodadas, o Opus sozinho nunca errou um teste. Ainda não medimos uma tarefa em que a qualidade varie.
- **Tarefas pequenas** (2–4 min): o custo fixo do líder pesa mais do que pesaria numa tarefa de horas.
- **Modelos misturados**: a equipe tem Haiku e Sonnet; parte da derrota é da velocidade do Haiku, não da orquestração em si.
- **Um fornecedor só**: todos os agentes medidos são Claude.
- **Correções guiadas pelos dados**: E–G nasceram das falhas de A–D, na mesma tarefa.
- **Ambiente único**: um contêiner, uma versão do Claude Code.

Próximos passos

	O que	Responde a
1	**Tarefa difícil**, em que o agente sozinho fique abaixo de $\sim$70%	Q1 (qualidade)
2	**Sonnet + revisor Opus contra Opus sozinho**: um modelo barato escrevendo e um forte revisando chega à mesma nota por menos?	Q1, Q3
3	**Montagem gerada**: com Automatic teams ligado, o líder escolhe "solo" quando deve?	Q3
4	**Revisor de outro fornecedor** (Codex revisando Claude), a vantagem própria da D01	Q1, tese
5	**Mais rodadas** ($\geq$ 8 por montagem) para chegar à significância	todas
6	Tirar o líder do caminho entre as fases (tarefas com dependências, O8)	Q2

Contribuições até aqui

1. Um **benchmark reprodutível** de orquestração (tarefas inventadas, testes escondidos, rodadas zeradas, ordem contrabalançada) com **grupo de controle de agente sozinho**.
2. Evidência [medido] de que, em duas tarefas de programação, um agente forte sozinho foi **mais rápido e mais barato** que a equipe, com a mesma nota, **inclusive numa tarefa paralelizável**, contra a previsão tirada da literatura.
3. Uma **regra de decisão** (D22) e uma ferramenta que a aplica: gerador por formatos conhecidos, com portão e aceite verificado pelo ambiente.
4. Um **protocolo de orquestração agnóstico** ao fornecedor, que não exige cooperação do agente, e o método para medi-lo contra um oráculo.
5. Um registro de 24 decisões com alternativas, evidência classificada e condição de revisão.

Perguntas para o orientador

- O **recorte** "quando a orquestração compensa" está bom, com o NarraTer como instrumento e não como produto final?
- A fase exploratória (protocolo, leitor de estado) entra no texto como capítulo próprio ou como apêndice?
- Quantas rodadas por montagem e que nível de significância são esperados?
- Uma **tarefa difícil** é obrigatória para sustentar a conclusão, ou o resultado "no teto" já basta, bem delimitado?
- A base teórica (MAST, Kim et al., Vals.ai, Tran e Kiela) é suficiente, ou falta alguma linha?
- Como declarar o uso de um agente de IA no desenvolvimento e na análise?

Obrigado

Código, dados brutos, relatório completo e como reproduzir:
`docs/relatorios/` (`orquestracao-versoes`, `quando-orquestrar`, `equipe-ou-sozinho`)
`bench/orquestracao/` `docs/decisoes.md` `docs/pesquisa/orquestracao.md`

Apêndice: como reproduzir

Leitor de estado (Estudo 1):

```
cd src-tauri && cargo run --release --example turn_eval -- --reps 5 --claude
cd src-tauri && cargo run --example turn_replay -- GRAVACAO.cast --at 8
cd src-tauri && cargo test turn_transcripts
```

Benchmark (Estudo 2):

```
git worktree add --detach $BENCH/vX <commit>      # uma por versão
$BENCH/build.sh A B C D E F G                       # compila cada versão
Xvfb :77 -screen 0 1600x1000x24 &
cd bench/orquestracao && ./rodar_todas.sh           # rodadas zeradas, ordem contrabalançada
python3 analisar.py $BENCH/runs/v?-r[1-9] && python3 agregar.py
```

Equipe contra agente sozinho:

```
BENCH_VARIANTE=solo python3 rodar.py H 1                     # tarefa pedidos, sozinho
BENCH_TAREFA=armazem BENCH_VARIANTE=solo|equipe|revisor python3 rodar.py H 1
python3 solo.py && python3 armazem.py                         # tabelas dos relatórios
```

Métricas: tokens processados = entrada + *cache* lido + *cache* criado, por chamada à API, tirados das transcrições. Custo = preço de API equivalente (a conta usada é de assinatura).